

Take-Home Assignment: Internal Products — Full-Stack Engineer (Mid)

Company: TechKraft Inc.

Position: Full Stack Engineer (Mid)

Time Limit: 2.5 hours (150 minutes)

Format: Take-home, submit via public GitHub repository

Overview

You are building an internal candidate scoring and review dashboard for TechKraft's recruitment workflow. This is the kind of internal tool we build regularly — admin UIs, scoring systems, AI-assisted review interfaces.

Submission: Create a public GitHub repository with your solution and share the link.

Problem Statement

TechKraft's recruitment team needs a web-based tool to manage candidate assessments. Reviewers need to score candidates across categories and view AI-generated summaries. Admins need full visibility.

Core Requirements

1. Backend API (Python/FastAPI)

Endpoint	Description
GET /candidates	

Endpoint	Description
	List candidates with filters: status, role_applied, skill, keyword + pagination
GET /candidates/{id}	Candidate detail with scores and AI-generated summary
POST /candidates/{id}/scores	Submit a score for a category (score 1–5, category name, optional note)
POST /candidates/{id}/summary	Trigger mock AI summary generation — simulates an async LLM call (2s delay)
GET /candidates/{id}/stream	(Stretch goal) SSE endpoint that streams score updates in real time

Pagination: offset-based with configurable page size (default 20, max 50).

2. Database (DynamoDB-style or SQLite)

Model your data with these entities:

- **candidates:** id, name, email, role_applied, status (new/reviewed/hired/rejected), skills (list/array), internal_notes (admin-only), created_at
- **scores:** id, candidate_id, category, score (1–5), reviewer_id, note, created_at

Use appropriate indexes (e.g., on candidates.status, candidates.role_applied, scores.candidate_id).

3. Role-Based Access Control

- **JWT-based authentication** with email + password
- **reviewer role:** Can score candidates, sees only their own scores, cannot view internal_notes
- **admin role:** Can see all scores from all reviewers, can view and edit internal_notes
- Registration must hardcode role to reviewer — never accept role from the client

4. Frontend (React + Vite)

- Login page

- Candidate list page with filter controls (status, role, skill, keyword) and pagination
- Candidate detail page showing:
 - Profile info
 - Scores (reviewer sees own scores; admin sees all)
 - Scoring form (select category + score + note)
 - AI summary section (trigger via button, display loading state while generating)
- Admin-only internal notes panel

5. Containerization (Docker Compose)

Docker Compose setup that starts both services: - Backend (FastAPI on port 8000) - Frontend (Vite dev server or nginx static build on port 5173)

6. Testing

At least 2–3 tests covering: - An API endpoint test (e.g., create a candidate, verify response) - An auth enforcement test (e.g., reviewer cannot see another reviewer's scores)

Debugging Signal

The following query pattern has a subtle bug. In your README, identify the issue, explain why it matters at scale, and describe the correct approach:

```
# from a hypothetical service layer – what's wrong here?
def search_candidates(status: str, keyword: str, page: int,
    page_size: int):
    all_candidates = db.execute("SELECT * FROM
        candidates").fetchall()
    filtered = [c for c in all_candidates if c["status"] ==
        status]
    # ... also filter by keyword in Python ...
    offset = (page - 1) * page_size
    return filtered[offset : offset + page_size]
```

Architecture Decision Record

In your README, include a brief ADR section documenting 2–3 key technical decisions. For each: - **Context:** What was the situation? - **Decision:** What did you choose? - **Trade-off:** What did you accept?

Examples: why FastAPI over alternatives, why this DB schema shape, how auth is handled.

Learning Reflection

In your README, add 2–3 sentences on one thing you tried for the first time or one approach you'd explore given more time.

Responsibility & Detail Checks

These are automatically verified by the evaluator: - Credentials must NOT be committed (use `.env.example` with dummy values) - README commands and port numbers must match the actual system - The mock AI summary endpoint must show loading/error states in the frontend (not just a blank page while waiting) - If a candidate is deleted, it must be a soft delete (set `status = "archived"` or add `deleted_at`) — never hard-delete

Evaluation Criteria

Category	Weight	What We're Looking For
Problem Solving	15%	Filter/pagination design, API structure, handling edge cases
Core Technical	15%	FastAPI async patterns, React component architecture, DB schema + indexing
System Design	10%	ADR quality, data flow reasoning, serverless-aware design thinking
Code Quality	10%	Code organization, error handling, naming, separation of concerns
Debugging	10%	Correctly identifying the bug snippet issue and explaining the fix
Practical Experience	5%	Docker Compose setup, env config, real-world patterns (soft delete, error handling)
Communication	5%	README clarity, ADR explanations, setup instructions
Ownership	10%	All core requirements completed, limitations honestly acknowledged
Learning	10%	

Category	Weight	What We're Looking For
		Learning reflection quality, tool choice reasoning
Culture Fit	5%	Testing included, security awareness (no credentials, no role spoofing)
Responsibility & Detail	5%	Docs match system, no committed credentials, subtle constraints met, AI/human tool use verified

Automatic Caps

- role accepted from client at registration → Responsibility & Detail capped at 2
- .env with real credentials committed → Responsibility & Detail capped at 2
- README port/command contradicts actual system → Responsibility & Detail capped at 2
- Core requirement absent with no acknowledgement → Responsibility & Detail capped at 2

Expected Directory Structure

```

/
├── README.md
├── docker-compose.yml
├── backend/
│   ├── Dockerfile
│   ├── app/
│   │   ├── main.py
│   │   ├── models.py
│   │   ├── schemas.py
│   │   ├── auth.py
│   │   ├── routers/
│   │   │   ├── candidates.py
│   │   │   └── auth.py
│   │   └── services/
│   │       └── candidate_service.py
│   ├── tests/
│   │   └── test_api.py
│   └── requirements.txt
└── frontend/

```

```
|   |— Dockerfile
|   |— src/
|   |   |— App.jsx
|   |   |— components/
|   |   |— pages/
|   |   └─ api/
|   └─ package.json
|   └─ vite.config.js
└─ .env.example
```

Tips

- Focus on quality over quantity — a complete, working submission with fewer features beats a broken one with everything attempted
 - Run your Docker Compose setup and test at least one curl command before submitting
 - Explain trade-offs honestly in your ADR
 - Make sure your README port numbers match your Docker Compose configuration
 - The LLM endpoint is a mock — show you understand how async external API calls should work
 - If you use AI tools, verify every line before submitting
-

Submission Instructions

1. Create a public GitHub repository
 2. Push your code
 3. Share the repository URL
 4. Ensure README.md is at the root with:
 - Setup and run instructions
 - ADR section (2–3 decisions)
 - Debugging bug identification
 - Learning reflection
 - Example API calls (curl commands)
-

Good luck. We're looking forward to seeing your approach to building the kind of internal tools our team ships every day.